Python Selenium + BDD (Behave) E2E Interview Q&A

---

1. What is Selenium?

Selenium is an open-source automation tool for web applications. It consists of WebDriver, IDE, Grid, and previously RC (deprecated).

2. What is BDD?

Behavior-Driven Development (BDD) is an agile approach using Gherkin syntax for plain English test scenarios to improve communication between developers, testers, and stakeholders.

3. How do you integrate Selenium with Behave?

Project structure:

```
features/
  login.feature
  steps/
    login_steps.py
  environment.py
```

Install:

```
pip install selenium behave
```

```
from selenium import webdriver
from behave import given, when, then
from selenium.webdriver.common.by import By

@given('user is on login page')
```

4. Example Feature File (Gherkin)

```
Feature: Login functionality
  Scenario: Successful login with valid credentials
    Given user is on login page
    When user enters valid username and password
    Then user should be redirected to homepage
```

5. Step Definitions Example

```
def step_impl(context):
    context.driver = webdriver.Chrome()
    context.driver.get('https://example.com/login')

@when('user enters valid username and password')
def step_impl(context):
    context.driver.find_element(By.ID, 'username').send_keys('testuser')
    context.driver.find_element(By.ID, 'password').send_keys('password')
    context.driver.find_element(By.ID, 'login').click()

@then('user should be redirected to homepage')
def step_impl(context):
    assert 'Home' in context.driver.title
    context.driver.quit()
```

6. environment.py Example

```
def before_scenario(context, scenario):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

def after_scenario(context, scenario):
    context.driver.quit()
```

7. Dynamic Locators in Steps

```
When user enters username "<username>" and password "<password>"
```

```
@when('user enters username "{username}" and password "{password}"') def
step_impl(context, username, password):
    context.driver.find_element(By.ID, 'username').send_keys(username)
    context.driver.find_element(By.ID, 'password').send_keys(password)
```

8. Explicit Wait Example

9. Handling Dropdowns, Checkboxes, Radio Buttons

```
from selenium.webdriver.support.ui import Select

# Dropdown
dropdown = Select(context.driver.find_element(By.ID, 'dropdown_id'))
dropdown.select_by_visible_text('Option 1')

# Checkbox
checkbox = context.driver.find_element(By.ID, 'checkbox_id') if not
checkbox.is_selected():
    checkbox.click()
```

10. Multiple Windows Example

```
main_window = context.driver.current_window_handle
all_windows = context.driver.window_handles
context.driver.switch_to.window(all_windows[1])
```

11. Alerts Example

```
alert = context.driver.switch_to.alert
alert.accept()
```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

```
WebDriverWait(context.driver, 10).until(
EC.visibility_of_element_located((By.ID, 'welcome_msg'))
)
```

```
# login_page.py
from selenium.webdriver.common.by import By
class LoginPage:
```

12. Data-Driven Tests (Scenario Outline)

```
Scenario Outline: Login with multiple credentials
  Given user is on login page
  When user enters username "<username>" and password "<password>"
  Then user should see "<result>"

Examples:
  | username | password | result        |
  | user1    | pass1    | Home page     |
  | user2    | pass2    | Invalid login |
```

13. Page Object Model (POM) Example

```
    def __init__(self, driver):
        self.driver = driver
        self.username = driver.find_element(By.ID, 'username')
        self.password = driver.find_element(By.ID, 'password')
        self.login_button = driver.find_element(By.ID, 'login')

    def login(self, user, pwd):
        self.username.send_keys(user)
        self.password.send_keys(pwd)
        self.login_button.click()
```

14. Using POM in Steps

```
from pages.login_page import LoginPage

@given('user logs in with valid credentials')
def step_impl(context):
    page = LoginPage(context.driver)
    page.login('testuser', 'password')
```

15. Screenshots on Failure

```
def after_scenario(context, scenario):
    if scenario.status == 'failed':
        context.driver.save_screenshot(f'screenshots/{scenario.name}.png')
    context.driver.quit()
```

16. File Upload Example

```
context.driver.find_element(By.ID, 'file_upload').send_keys('C:/path/file.txt')
```

17. JavaScript Execution

```
context.driver.execute_script('window.scrollTo(0, 1000)')
context.driver.execute_script('arguments[0].click();', element)
```

18. Reporting with Allure

19. CI/CD Integration (Jenkins)

1. Clone repository
2. Install dependencies: pip install -r requirements.txt
3. Run tests: behave -f allure_behave.formatter:AllureFormatter -o reports/
4. Publish Allure reports in Jenkins

20. End-to-End Strategy

1. Analyze requirements Identify scenarios
2. Write **Gherkin feature files**
3. Implement **step definitions** using Selenium WebDriver
4. Use **POM** for maintainable page objects
5. Implement **waits, dynamic locators, alerts, frames, windows**
6. Handle **data-driven scenarios** via Scenario Outline
7. Run tests locally or in **parallel using Selenium Grid + Behave**
8. Generate **Allure/HTML reports**
9. Integrate with **CI/CD pipelines (Jenkins/GitHub Actions)**
10. Maintain and update tests as the application evolves

```
pip install allure-behave
behave -f allure_behave.formatter:AllureFormatter -o reports/
```

21. How do you perform mouse hover actions in Selenium?

Answer:

```
from selenium.webdriver import ActionChains
element = context.driver.find_element(By.ID, "hover_element")
action = ActionChains(context.driver)
action.move_to_element(element).perform()
```

---

22. How do you perform drag-and-drop?

Answer:

```
source = context.driver.find_element(By.ID, "source")
target = context.driver.find_element(By.ID, "target")
action = ActionChains(context.driver)
action.drag_and_drop(source, target).perform()
```

---

23. How do you handle stale element reference errors?

Answer:

- Use **explicit waits** or **re-locate the element** before performing actions.

```
element = WebDriverWait(context.driver, 10).until(
   EC.presence_of_element_located((By.ID, "element_id"))
)
```

---

24. How do you handle iframes?

Answer:

```
context.driver.switch_to.frame("frame_name")
# perform actions inside iframe
context.driver.switch_to.default_content()
```

---

25. How do you check if an element is visible or enabled?

Answer:

```
element = context.driver.find_element(By.ID, "element_id")
element.is_displayed()  # True/False
element.is_enabled()    # True/False
```

---

26. How do you perform keyboard events?

Answer:

```
from selenium.webdriver.common.keys import Keys
element = context.driver.find_element(By.ID, "input")
element.send_keys(Keys.ENTER)
element.send_keys(Keys.TAB)
```

---

27. How do you handle broken links?

Answer:

```
import requests
links = context.driver.find_elements(By.TAG_NAME, "a")
for link in links:
   url = link.get_attribute("href")
   if url:
      r = requests.head(url)
      if r.status_code >= 400:
         print(f"Broken link: {url}")
```

---

28. How do you scroll to an element?

Answer:

```
element = context.driver.find_element(By.ID, "target")
context.driver.execute_script("arguments[0].scrollIntoView();", element)
```

---

29. How do you perform double-click or right-click?

Answer:

```
action = ActionChains(context.driver)
element = context.driver.find_element(By.ID, "button")
action.double_click(element).perform()
action.context_click(element).perform()
```

---

30. How do you switch between multiple tabs?

Answer:

```
tabs = context.driver.window_handles
context.driver.switch_to.window(tabs[1])
```

---

31. How do you handle dynamic tables?

Answer:

```
rows = context.driver.find_elements(By.XPATH, "//table[@id='table']/tbody/tr")
for row in rows:
   cells = row.find_elements(By.TAG_NAME, "td")
   for cell in cells:
      print(cell.text)
```

---

32. How do you handle tooltips?

Answer:

```
from selenium.webdriver.common.action_chains import ActionChains
element = context.driver.find_element(By.ID, "tooltip")
action = ActionChains(context.driver)
action.move_to_element(element).perform()
tooltip_text = element.get_attribute("title")
```

---

33. How do you wait for JavaScript or AJAX to load?

Answer:

```
WebDriverWait(context.driver, 10).until(
   lambda driver: driver.execute_script("return jQuery.active == 0")
)
```

---

34. How do you take full-page screenshots?

Answer:

```
context.driver.save_screenshot("fullpage.png")
```

---

35. How do you take screenshot of an element only?

Answer:

```
element = context.driver.find_element(By.ID, "element")
element.screenshot("element.png")
```

---

36. How do you handle JavaScript alerts with text input?

Answer:

```
alert = context.driver.switch_to.alert
alert.send_keys("Some text")
alert.accept()
```

---

37. How do you handle browser navigation (back, forward, refresh)?

Answer:

```
context.driver.back()
context.driver.forward()
context.driver.refresh()
```

---

38. How do you maximize or minimize the browser window?

Answer:

```
context.driver.maximize_window()
context.driver.minimize_window()
```

---

39. How do you clear input fields?

Answer:

```
element = context.driver.find_element(By.ID, "input")
element.clear()
```

---

40. How do you get text, attributes, or tag name of elements?

Answer:

```
element = context.driver.find_element(By.ID, "element")
text = element.text
attr = element.get_attribute("href")
tag = element.tag_name
```

---

41. How do you perform conditional checks on elements?

Answer:

```
element = context.driver.find_element(By.ID, "checkbox")
if element.is_selected():
   print("Checkbox is already selected")
else:
   element.click()
```

---

42. How do you perform browser-specific options?

Answer:

```
from selenium.webdriver.chrome.options import Options
options = Options()
options.add_argument("--headless")
driver = webdriver.Chrome(options=options)
```

---

43. How do you handle SSL certificate warnings?

Answer:

```
options = webdriver.ChromeOptions()
options.add_argument('--ignore-certificate-errors')
driver = webdriver.Chrome(options=options)
```

---

44. How do you handle cookies?

Answer:

```
context.driver.add_cookie({"name": "test", "value": "123"})
print(context.driver.get_cookies())
context.driver.delete_cookie("test")
context.driver.delete_all_cookies()
```

---

45. How do you handle file downloads automatically?

Answer:

- Set browser preferences to download automatically:

```
options = webdriver.ChromeOptions()
prefs = {"download.default_directory": "C:/Downloads"}
options.add_experimental_option("prefs", prefs)
driver = webdriver.Chrome(options=options)
```

---

46. How do you run tests in parallel using Behave?

Answer:

- Use behave-parallel or run multiple scenarios with multiprocessing.

```
behave -f pretty -n "scenario_name" --processes 4
```

---

47. How do you integrate Behave tests with Jenkins?

Answer:

1. Clone repo in Jenkins job.
2. Install dependencies: pip install -r requirements.txt
3. Run Behave tests: behave -f allure_behave.formatter:AllureFormatter -o reports/
4. Publish Allure reports in Jenkins.

---

48. How do you handle Captcha in Selenium tests?

Answer:

- Selenium **cannot solve Captcha automatically**.
- Options: manual intervention or third-party services like 2Captcha/AntiCaptcha.

---

49. How do you handle dynamic dropdowns (autocomplete)?

Answer:

```
context.driver.find_element(By.ID, "search").send_keys("Option")
WebDriverWait(context.driver, 10).until(
   EC.visibility_of_element_located((By.XPATH, "//li[contains(text(),'Option')]"))
).click()
```

---

50. How do you design an end-to-end automation strategy?

Answer:

1. Analyze requirements and identify scenarios.
2. Write **feature files** in Gherkin.
3. Implement **step definitions** using Selenium WebDriver.
4. Use **POM** for page objects.
5. Handle waits, dynamic locators, frames, alerts, windows.
6. Implement **data-driven testing** via Scenario Outline.
7. Run tests locally or in parallel (Selenium Grid).

8. Generate **Allure/HTML reports**.
9. Integrate with **CI/CD (Jenkins/GitHub Actions)**.
10. Maintain tests as application evolves.